Lecture 06 – Classification

Stat 5376: Statistical Machine Learning

Ruiqi Liu

Department of Mathematics and Statistics
Texas Tech University

http://www.math.ttu.edu/~ruiqliu/

Disclaimer: These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.

Categorical/Qualitative data

It is often of interest to model categorical data (as response variables)

- Categorical data: a categorical variable has a measurement scale that consists of a set of categories;
  - Attitude toward gun control: favor or oppose;(nominal)
  - Gender: male or female;(nominal)
  - Education: Did not graduate from high school, high school, some college, bachelor's degree, master's degree, Ph. D.;(ordinal)
  - Cholesterol level (high, moderate, low);(ordinal)
  - Heart disease (yes, no);(nominal)

Categorical/Qualitative data

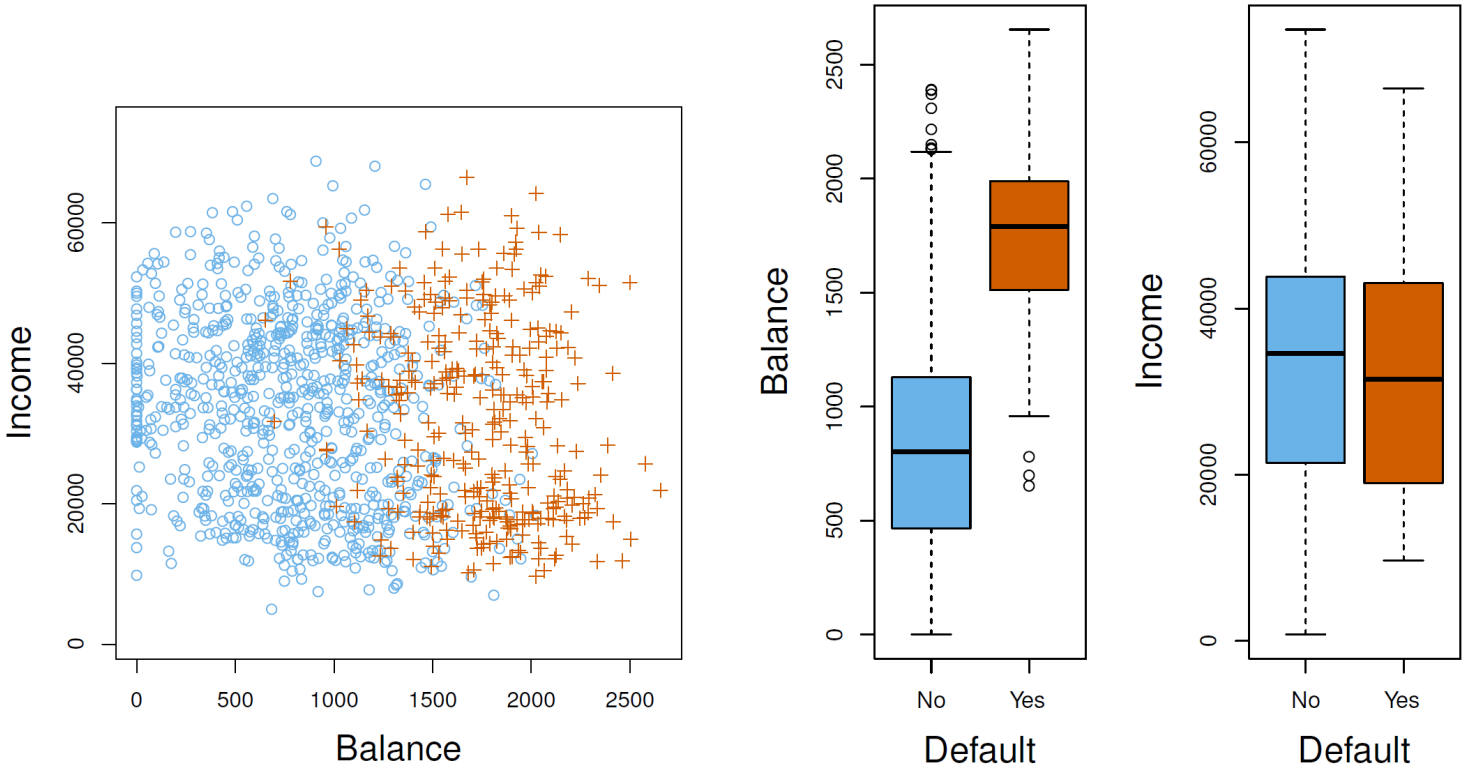
- In 1954, 401,974 children were participants in a large clinical trial for polio vaccine. Of these 201,229 were given the vaccine, and 200,745 were given a placebo. 110 of the children who received a placebo got polio and 33 of those given the vaccine got polio. Was the vaccine effective?
- Stillbirth is the death of a fetus at any time after the twentieth week of pregnancy. A premature birth is the live birth of a child from the twentieth until the thirty-seventh week of pregnancy. Fit loglinear models to investigate the association among the following variables that were recorded in a study of stillbirth in the Australian state of Queensland:
  - Birth status(B) - stillbirth or live birth; (response)
  - Gender(G) - male or female;
  - Gestational age(A) $\leq$ 24, 25-28, 29-32, 33-36, 37-41 weeks;
  - Race(R) - Aborigine or white;
- 59 alligators were sampled in Florida. The response is primary food type: Fish, Invertebrate, and Other. The explanatory variable is length of the alligator in meters.

Typical Classification Problem

- Suppose the categorical response variable Y belongs to a set $\mathcal{C}$ such as:
  - eye color $\in$ {brown, blue, green};
  - insurance claim $\in$ {fraud, not fraud}.
- We would have additional information X (a feature vector) related to each Y;
- The goal of classification:
  - To develop a decision rule $C : X \to \mathcal{C}$;
  - Predict value for Y: $\hat{Y} = C(X) \in \mathcal{C}$;
  - The decision rule $C(X)$ is termed as a $\mathbf{Classifier}$.
- It is often desirable to find the probabilities of $Y \in \mathcal{C}$ given the feature vector X.

Example: Credit Card Defualt

- Response Y: a credit card holder will default or not?
- Feature vector X: (the balance of the credit card, personal income);



Can We Use Linear Regression?

- Suppose we could code

$$Y = \begin{cases} 0 & \text{if No,} \\ 1 & \text{if Yes.} \end{cases}$$

- Can we run a linear regression

$$Y = X^T \boldsymbol{\beta} + \boldsymbol{\epsilon}$$

and classify as “Yes" if $\widehat{Y} = X^T \widehat{\boldsymbol{\beta}}_{ols} > 0.5$?

  - In this case, the answer is yes since Y has only two categories;
  - It is equivalent to linear discriminant analysis (LDA); (we will talk about later)
  - For binary data, we have $E(Y|X = x) = Pr(Y = 1|X = x)$; therefore, $\widehat{Y}$ from the linear regression model provides an estimator for $Pr(Y = 1|X = x)$;
  - However, $\widehat{Y}$ is not guaranteed to be between 0 and 1; not good for probabilities!

Can We Use Linear Regression?

- Now suppose we have a response variable with three possible values. A patient presents at the emergency room, and we must classify them according to their symptoms.

$$Y = \begin{cases} 1 & \text{if stroke,} \\ 2 & \text{if drug overdose,} \\ 3 & \text{if epileptic seizure.} \end{cases}$$

- This coding suggests an ordering, implies that the difference between "stroke" and "drug overdose" is the same as between "drug overdose" and "epileptic seizure".
- Linear regression is not appropriate here.
- *Multinomial Logistic Regression* is more appropriate.

Multinomial Logistic Model

- Suppose there are K classes.
- Let $Y \in \{1, \ldots, K\}$ be the label and $\mathbf{X} \in \mathbb{R}^p$ be the covariates.
- For $k \in \{1, \ldots, K\}$, the Multinomial Logistic Model is

$$P(Y = k|\mathbf{X} = x) = \frac{e^{\beta_{0k} + x^T \beta_k}}{\sum_{j=1}^{K} e^{\beta_{0j} + x^T \beta_j}}.$$

- However, this problem is over-identified.
- For example, if we $(\beta_{02}, \beta_2), \ldots, (\beta_{0p}, \beta_p)$ are known, according to the equation $\sum_{j=1}^{p} P(Y = k|\mathbf{X} = x) = 1$, we can calculate (β_{01}, β_1).
- Solution: specify a reference class, say class 1, and the model becomes

$$P(Y = k|\mathbf{X} = x) = \begin{cases} \frac{1}{1+\sum_{j=2}^{K} e^{\beta_{0j} + x^T \beta_j}} & \text{if k=1 ;} \\ \frac{e^{\beta_{0k} + x^T \beta_k}}{1+\sum_{j=2}^{K} e^{\beta_{0j} + x^T \beta_j}} & \text{if k=2,. . . , p ;} \end{cases}$$

- There are totally $(K - 1) \times (p + 1)$ parameters.

Maximal Likelihood Estimation

- The MLE maximizes $L(\boldsymbol{\beta})$ as follows:

$$\sum_{i=1}^{n} \log \left\{ \left(\frac{1}{1+\sum_{j=2}^{K} e^{\beta_{0j}+x_i^T\beta_j}} \right)^{I(y_i=1)} \times \prod_{k=2}^{K} \left(\frac{e^{\beta_{0k}+x_i^T\beta_k}}{1+\sum_{j=2}^{K} e^{\beta_{0j}+x_i^T\beta_j}} \right)^{I(y_i=k)} \right\}$$

- Let $\widehat{\boldsymbol{\beta}} = (\widehat{\beta}_{02}, \widehat{\beta}_2, \ldots, \widehat{\beta}_{0K}, \widehat{\beta}_K)$ be the MLE.
- The log likelihood is defined as $\mathcal{L} = L(\widehat{\boldsymbol{\beta}}) < 0$.
- The deviance is defined as $\mathcal{D} = -2\mathcal{L} > 0$.
- There are totally $(K-1) \times (p+1)$ parameters.

Logistic Regression with R

- R package "nnet".
- Dataset "wine" contains three categories.
-
```
library(rattle.data)
library(nnet)
data(wine)
lmod1=multinom(Type~Alcohol+Color,data=wine)
summary(lmod1)

Coefficients:
  (Intercept)   Alcohol      Color
2    66.31584 -4.650077 -1.2995713
3    37.87299 -3.219394  0.8376598

Std. Errors:
  (Intercept)   Alcohol     Color
2   11.964377 0.8949827 0.4354904
3    8.419196 0.6627470 0.1828222

Residual Deviance: 141.8154
AIC: 153.8154
```

Logistic Regression with R

- Calculating pvalue

```
z=summary(lmod1)$coefficients/summary(lmod1)$standard.errors
# 2-tailed Wald z tests to test significance of coefficients
p=(1 - pnorm(abs(z), 0, 1)) * 2
p
```

```
   (Intercept)      Alcohol        Color
2 2.977164e-08 2.039318e-07 2.843619e-03
3 6.846364e-06 1.187865e-06 4.609314e-06
```

- It seems okay. All coefficients are significant.

Likelihood Ratio Test and Deviance

- To compare two nested models M_1 and M_2, say $M_1 \subset M_2$, that is, M_1 is a special case of M_2;
- The LRT statistic is used for testing that the simpler model M_1 holds;
- The IRT compares difference in maximized $\log$-likelihood:

$$Q_L = 2\{\mathcal{L}(M_2) - \mathcal{L}(M_1)\}$$

- It is easily shown that

$$Q_L = 2\{\mathcal{L}(M_2) - \mathcal{L}(M_1)\} = D(M_1) - D(M_2)$$

- Assuming that the
  - M_1 is equivalent to M_2
  - the distributional assumptions hold

  when $n \to \infty$,

$$Q_L \to^d \chi^2(d);$$

- d is the difference in # of parameters of model M_1 and M_2.

Logistic Regression with R

- M_1 : Alcohol+Color
- M_2 : Alcohol+Color+Hue

```
lmod1=multinom(Type~Alcohol+Color,data=wine)
lmod2=multinom(Type~Alcohol+Color+Hue,data=wine)
Q=lmod1$deviance-lmod2$deviance
Q

[1] 61.05241

pchisq(Q, df=1, lower.tail = F)

[1] 5.557542e-15

summary(lmod2)

Coefficients:
  (Intercept)  Alcohol      Color         Hue
2    62.85574 -4.30133 -1.6110485   0.3239058
3    40.14545 -1.99682  0.5317726 -19.0253877

Std. Errors:
  (Intercept)   Alcohol     Color      Hue
2    13.17812 0.9917615 0.5163967 2.921062
3    11.59869 0.9017044 0.3872225 4.672489
```

- pvalue< 0.05, reject null hypothesis $M_1 = M_2$. Model M_2 is better.

Saturated Model

- The saturated model is the most complicated model for multinomial regression.
- The saturated model will not lose any information.
- But it is to complicated to interpret the results.
- Given $(y_1, x_1), \ldots, (y_n, x_n)$, it assumes y_i is multinomial random variable with parameter $(p_{i1}, \ldots, p_{iK})$.
- So the MLE maximize $L(p_{11}, \ldots, p_{nK})$ as follows:

$$\log(\prod_{i=1}^{n} \prod_{k=1}^{K} p_{ik}^{I(y_i=k)})$$

- Clearly, the solution is $\widehat{p}_{ik} = I(y_i = k)$.
  - If $y_i = k$, then MLE guesses $P(y_i = k) = 1$.
- The log likelihood is defined as $\mathcal{L} = L(\widehat{p}_{11}, \ldots, \widehat{p}_{nK}) = 0$.
- The deviance is defined as $\mathcal{D} = -\mathcal{L} = 0$.
- There are totally n parameters.

Null Model

- The null model is the simplest model for multinomial regression, which only includes intercept.
- The MLE maximize $L(\boldsymbol{\beta})$ as follows:

$$\sum_{i=1}^{n} \log \left\{ \left(\frac{1}{1+\sum_{j=2}^{K} e^{\beta_{0j}}} \right)^{I(y_i=1)} \times \prod_{k=2}^{K} \left(\frac{e^{\beta_{0k}}}{1+\sum_{j=2}^{K} e^{\beta_{0j}}} \right)^{I(y_i=k)} \right\}$$

- Let $\widehat{\boldsymbol{\beta}} = (\widehat{\beta}_{02}, \ldots, \widehat{\beta}_{0K})$ be the MLE.
- The log likelihood is defined as $\mathcal{L} = L(\widehat{\boldsymbol{\beta}}) < 0$.
- The deviance is defined as $\mathcal{D} = -\mathcal{L} > 0$.
- There are totally $K - 1$ parameters.

Compare with Saturated Model

- M_2 : Alcohol+Color+Hue
- Compare M_2 with the saturated model

```
lmod2=multinom(Type~Alcohol+Color+Hue,data=wine)
n=dim(wine)[1]
n

[1] 178

pchisq(lmod2$deviance, df=178-8, lower.tail = F)

[1] 1
```

- pvalue$= 1$, fail to reject null hypothesis $M_2 =$ saturated model. Model M_2 is equal to the saturated model.

Compare with Null Model

- M_2 : Alcohol+Color+Hue
- Compare M_2 with the null model

```
lmod2=multinom(Type~Alcohol+Color+Hue,data=wine)
lmod0=multinom(Type~1,data=wine)
pchisq(lmod0$deviance-lmod2$deviance, df=8-2, lower.tail = F)
```

```
[1] 4.524182e-63
```

- pvalue< 0.05, reject null hypothesis $M_2 =$ null model. Model M_2 is better than null model.

Information Criterions for Model Selection

- We have learned how to use “Deviance" and “LRT" to check “goodness of fit" and to compare nested models;
- But that does not cover all our needs;
- For example, how to compare modes that are not nested? How to do model selection?
- Note that the maximum likelihood estimator tries to minimize the Deviance, so the bigger the model, the smaller the deviance; (Much like RSS?)–Potential over-fitting problem! How to avoid?
- Suppose there are s parameters in the model
  - Akaike Information Criterion (AIC)

    $$AIC = \text{Deviance} + 2s = -2\ell_n(M) + 2s.$$

    where p is the number of parameters in the model;
  - Schwarz Criterion-A Bayesian argument yields the Bayesian information criterion: (BIC)

    $$BIC = \text{Deviance} + s\log(n) = -2\ell_n(M) + s\log(n).$$

AIC and BIC in R

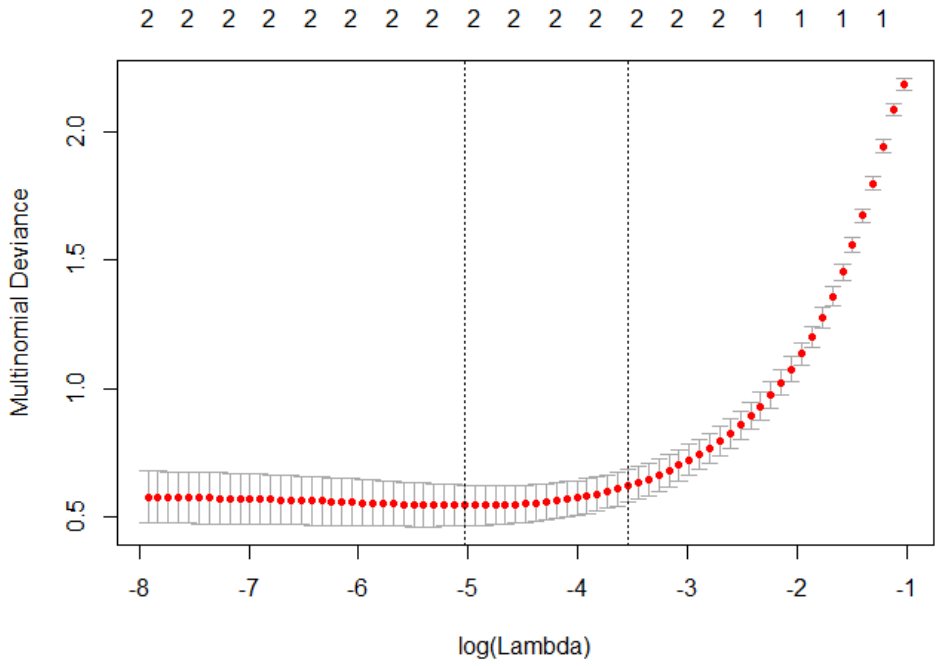
- M_2 : Alcohol+Color+Hue
- Totally $2 \times 3 = 6$ parameters
- Compute AIC and BIC of M_2.

```
lmod1=multinom(Type~Alcohol+Color,data=wine)
lmod1$AIC

[1] 153.8154

lmod1$deviance+2*6

[1] 153.8154

n=dim(wine)[1]
BIC=lmod1$deviance+6*log(n)
BIC

[1] 172.9061
```

Glmnet Package

- M_2 : Alcohol+Color+Hue
- Variable selection using lasso penalty.

```
y=wine$Type
x=wine[,c(2,11,12)]
x=as.matrix(x)
cv.lmod=cv.glmnet(x,y, family="multinomial")
plot(cv.lmod)

cf=coef(cv.lmod, s = "lambda.min")
cf
```



Glmnet Package

```
$`1`
4 x 1 sparse Matrix of class "dgCMatrix"
                     1
(Intercept) -28.318510
Alcohol       1.719862
Color         .
Hue           .

$`2`
4 x 1 sparse Matrix of class "dgCMatrix"
                    1
(Intercept) 22.822209
Alcohol     -1.812058
Color       -1.147687
Hue          .

$`3`
4 x 1 sparse Matrix of class "dgCMatrix"
                      1
(Intercept)   5.4963013
Alcohol       .
Color         0.3403836
Hue         -14.8154960
```

Glmnet Package

- The Glmnet does not use a reference class (it uses a different constrain).

$$P(Y=k|\mathbf{X}=x)=\frac{e^{\beta_{0k}+x^T\beta_k}}{\sum_{j=1}^K e^{\beta_{0j}+x^T\beta_j}}.$$

- Divided by $e^{\beta_{01}+x^T\beta_1}$ on top and bottom of the fractions, we can see

$$P(Y=k|\mathbf{X}=x)=\frac{e^{(\beta_{0k}-\beta_{01})+x^T(\beta_k-\beta_1)}}{\sum_{j=1}^K e^{(\beta_{0j}-\beta_{01})+x^T(\beta_j-\beta_1)}}.$$

- `cbind(cf[[2]]-cf[[1]],cf[[3]]-cf[[1]])`

```
4 x 2 sparse Matrix of class "dgCMatrix"
                     1           1
(Intercept) 51.140718  33.8148111
Alcohol     -3.531920  -1.7198620
Color       -1.147687   0.3403836
Hue          .        -14.8154960
```

Odd Ratio

- Consider multinomial logistic regression with $K = 2$

$$P(Y = k|\mathbf{X} = x) = \begin{cases} \frac{1}{1+e^{\beta_{02}+x^T\beta_2}} & \text{if k=1 ;} \\ \frac{e^{\beta_{02}+x^T\beta_2}}{1+e^{\beta_{02}+x^T\beta_2}} & \text{if k=2 ;} \end{cases}$$

- The odd ratio is defined as $odd_2(x) = \frac{P(Y=2|\mathbf{X}=x)}{P(Y=1|\mathbf{X}=x)} = e^{\beta_{02}+x^T\beta_2}$
- So $\log(odd_2(x)) = \beta_{02} + x^T\beta_2$
- Classify x to class 2 if $\log(odd_2(x)) \geq \log(odd_1(x)) = 1$
- Or equivalently, classify x to class 2 if

$$f(x) = b_{02} + x^T b_2 = \beta_{02} - 1 + x^T\beta_2 > 0.$$

- $f(x)$ can be viewed as a score for feature vector x that determines which class it belongs to.

What is a Hyperplane?

- The decision boundary is a hyperplane!
- In general the equation for a hyperplane has the form

$$\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = 0$$

  - In $p = 2$ dimensions a hyperplane is a line;
  - If $\beta_0 = 0$, the hyperplane goes through the origin, otherwise not;
- The hyperplane as dividing p-dimensional feature space into two halves
  - If the vector $x = (x_1, \cdots, x_p)^T$ satisfies

$$\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p > 0,$$

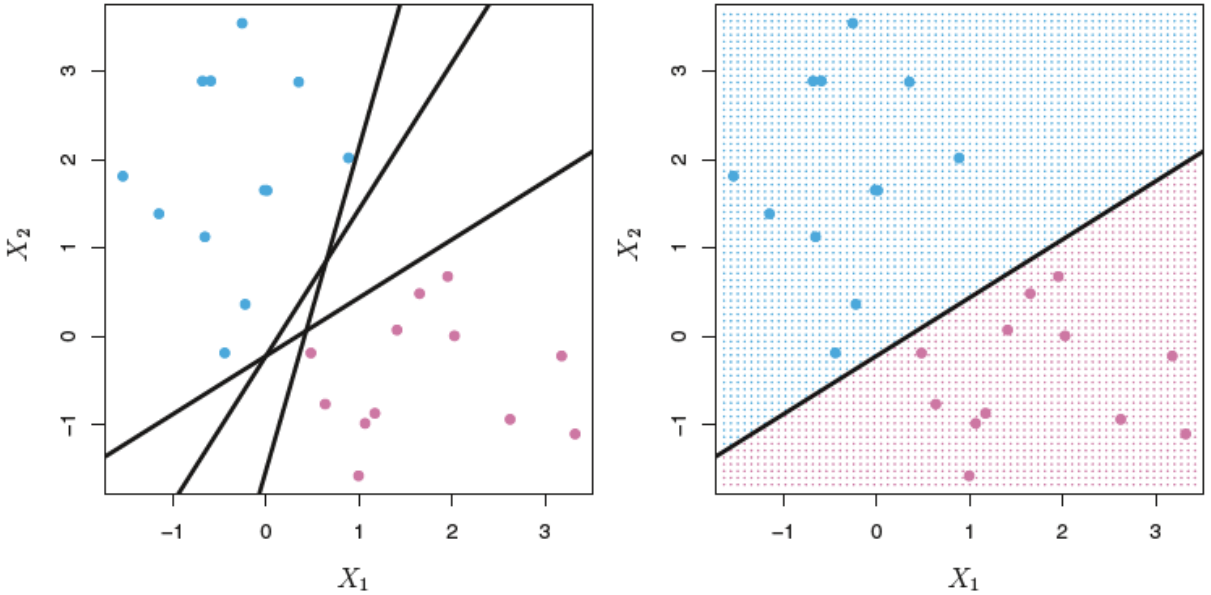
    then x lies to one side of the hyperplane;
  - If the vector $\boldsymbol{X} = (X_1, \cdots, X_p)^T$ satisfies

$$\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p < 0,$$

    then $\boldsymbol{X}$ lies on the other side of the hyperplane;

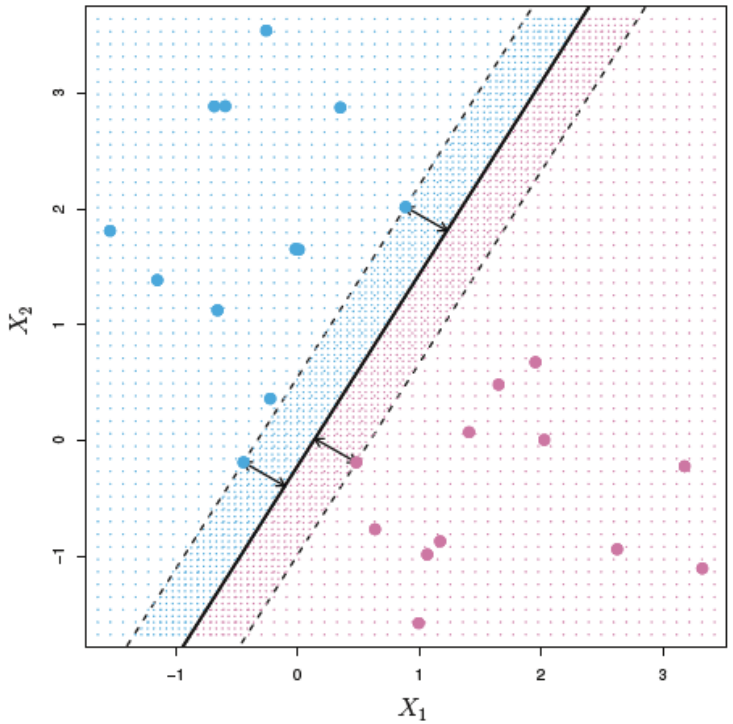
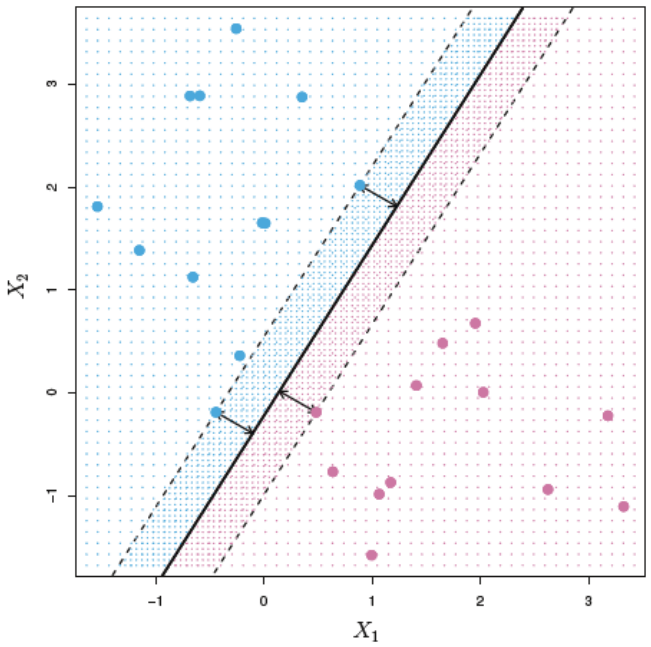
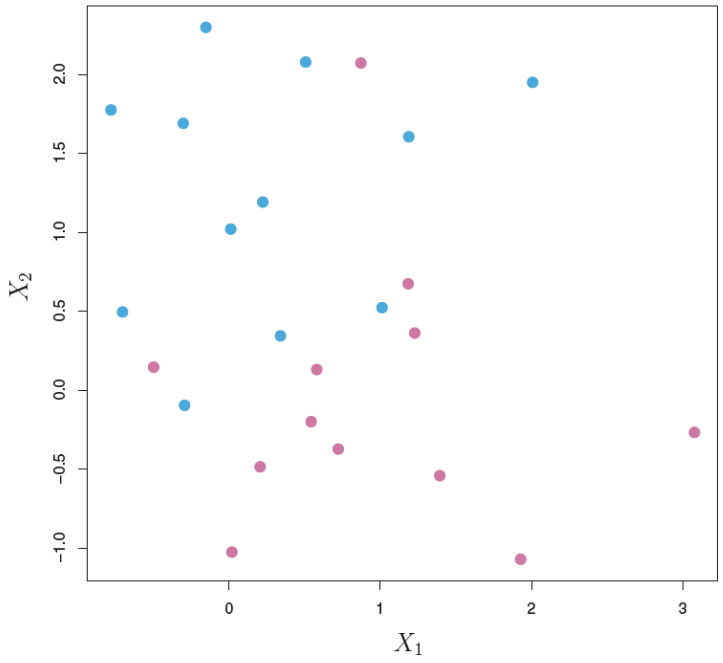
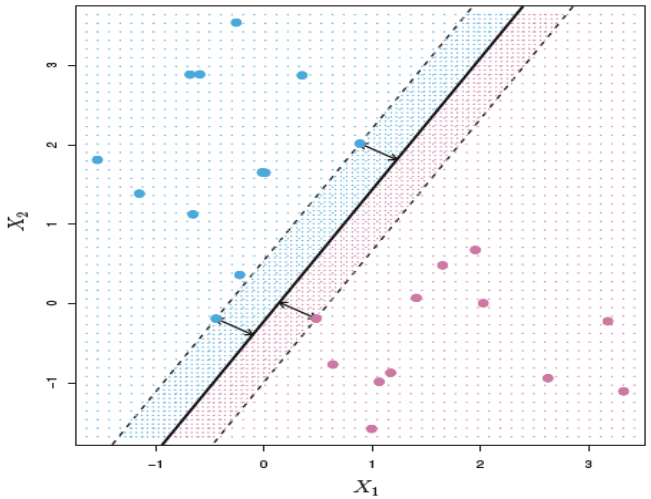
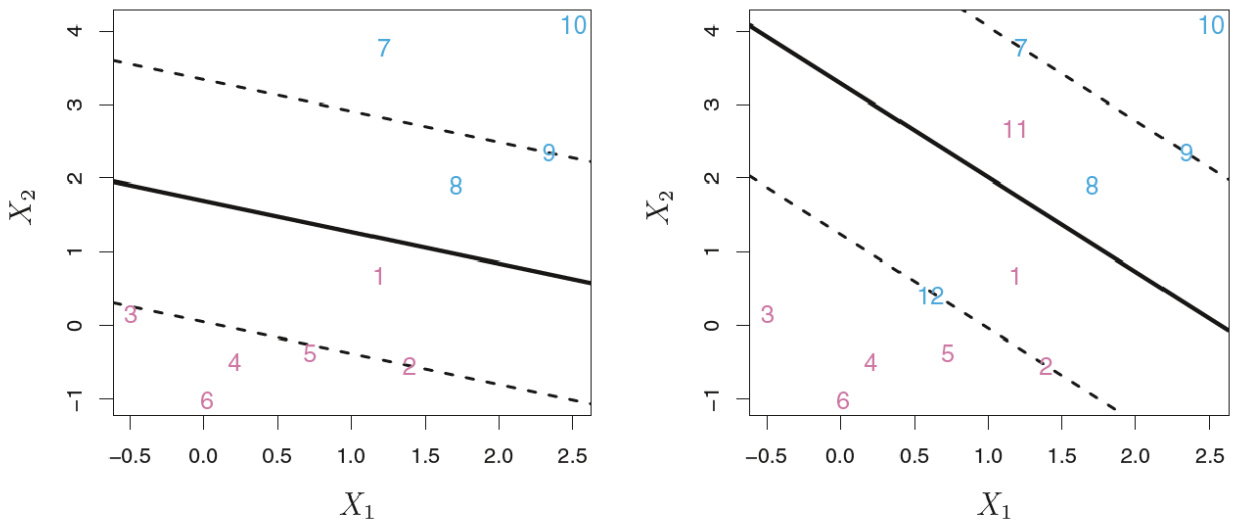
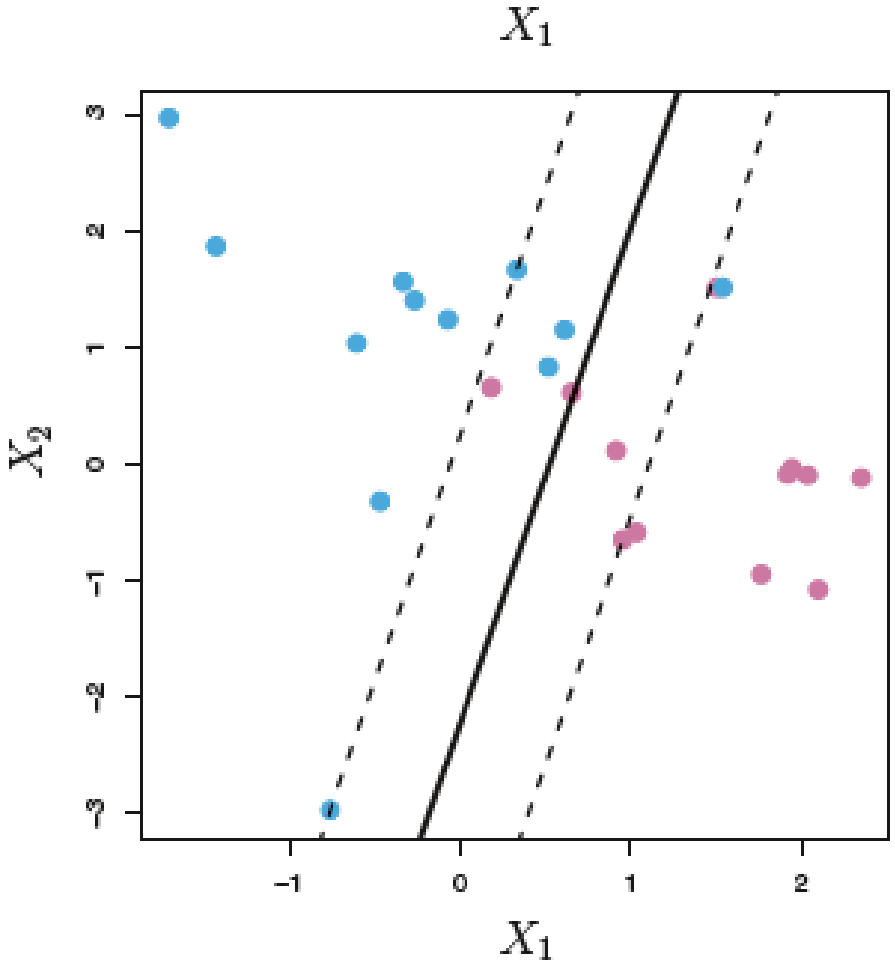
Separating Hyperplane

- When the data from two classes are perfectly separable, we can use hyperplane for classification!



- Let $f(\boldsymbol{X}) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$, then $f(\boldsymbol{X}) > 0$ for points on one side of the hyperplane, and $f(\boldsymbol{X}) < 0$ for points on the other;
- If we code the colored points as $Y_i = +1$ for blue, say, and $Y_i = -1$ for mauve, then we have $Y_i f(\boldsymbol{X}_i) > 0$ for any i;
- $f(\boldsymbol{X}) = 0$ defines a separating hyperplane!

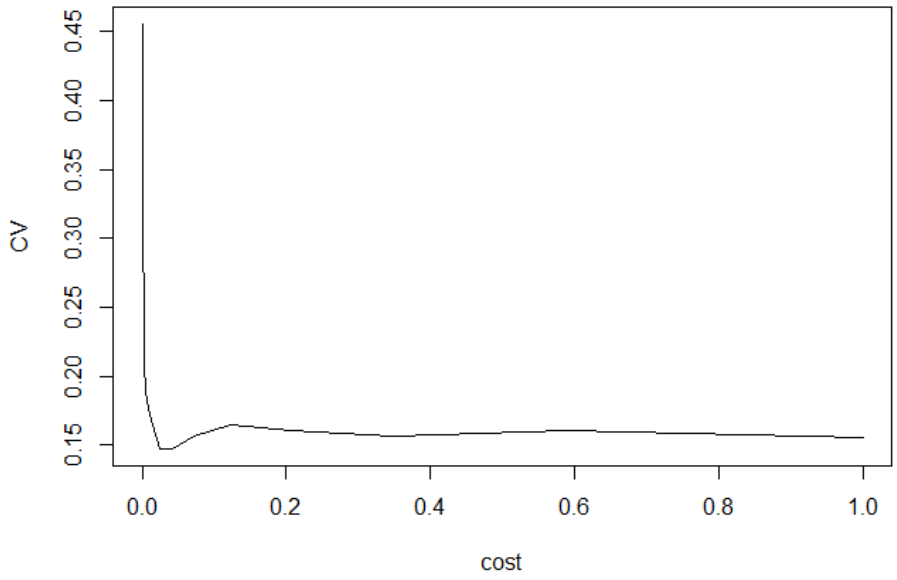
Perfect Separation in Logistic Regression

- ```
  n=100
  x=runif(n,-1,1)
  y=x+0.1
  y=y>0
  lmod=glm(y~x,family = "binomial")

  Warning messages:
  1: glm.fit: algorithm did not converge
  2: glm.fit: fitted probabilities numerically 0 or 1 occurred


  summary(lmod)

  Coefficients:
              Estimate Std. Error z value Pr(>|z|)
  (Intercept)    170.6    17447.0    0.01    0.992
  x             1774.6   180229.6    0.01    0.992

  (Dispersion parameter for binomial family taken to be 1)

      Null deviance: 1.3719e+02  on 99  degrees of freedom
  Residual deviance: 2.6064e-07  on 98  degrees of freedom
  AIC: 4
  ```
- Reason: $0.1 + x$, $170.6 + 1774.6x$ are both separating hyperplanes.
  ```

Maximal Margin Classifier

- There maybe infinite number of hyperplanes can perfectly separate the training data. Which one to use?
- Remember the “perfect separation" issue in the Logistic Regression?
- A new principle: choose the separating hyperplane that is farthest from the training observations.



- But how?

Maximal Margin Classifier

- For any separating hyper plane

$$f(\boldsymbol{X}) = b + \boldsymbol{X}^T w,$$

  - compute the (perpendicular) distances from all points $\boldsymbol{X}_1, \cdots, \boldsymbol{X}_n$, denoted as $d_1, \cdots, d_n$, where

$$d_i = \frac{|f(\boldsymbol{X}_i)|}{\|w\|} = \frac{|b + \boldsymbol{X}_i^T w|}{\sqrt{w^T w}}$$

  - Find the margin for $f(\boldsymbol{X})$ as $M = \min\{d_1, \cdots, d_n\}$
- For each (b, w), there is a different margin M, hence M is a function of $\boldsymbol{\beta}$;
- Maximal Margin Classifier: find the $\boldsymbol{\beta}$ such that the associated margin M is the largest!
- We hope that a classifier that has a large margin on the training data will also have a large margin on the test data, and hence will classify the test observations correctly;

Support Vectors

- The solid line: maximal margin hyperplane;
- The dashed line: the width of the margin;



- Support vectors: three training observations on the dashed line;
  - if these points were moved slightly then the maximal margin hyper plane would move as well;
  - the maximal margin hyperplane depends directly on the support vectors;
  - a movement to any of the other observations would not affect the separating hyperplane, provided that the observation's movement does not cause it to cross the boundary set by the margin (the dashed line).

Maximal Margin Classifier: Algorithm

- We have observations $(x_1, y_1), \cdots, (x_n, y_n)$;
- The class labels are defined as $y_i = +1$ or $y_i = -1$;
- We are looking for a hyperplane

$$f(x) = b + x^T w,$$

whose parameters $b \in \mathbb{R}, w \in \mathbb{R}^p$ by

$$\begin{aligned} \max_{b,w} M = \min_{i=1,\ldots,n} & \frac{|b + x_i^T w|}{\sqrt{w^T w}} \\ \text{subject to} \quad & y_i(b + x_i^T w) > 0 \text{ for any } i = 1, \cdots, n. \end{aligned}$$

- Since $y_i = -1, 1$, the constrain becomes $y_i(b + x_i^T w) = |b + x_i^T w|$.
- By definition of M, it is equivalent ot $y_i(b + x_i^T w) = |b + x_i^T w| \geq M\sqrt{w^T w}$

Maximal Margin Classifier: Algorithm

- The problem becomes

$$\max_{b,w} M = \min_{i=1,\ldots,n} \frac{|b + x_i^T w|}{\sqrt{w^T w}}$$
$$\text{subject to} \qquad y_i(b + x_i^T w) \geq M\sqrt{w^T w} \text{ for any } i = 1, \cdots, n.$$

- The problem is equivalent to

$$\max_{b,w} M = \min_{i=1,\ldots,n} \left| \frac{b}{\sqrt{w^T w}} + x_i^T \frac{w}{\sqrt{w^T w}} \right|$$
$$\text{subject to} \qquad y_i \left(\frac{b}{\sqrt{w^T w}} + x_i^T \frac{w}{\sqrt{w^T w}} \right) \geq M \text{ for any } i = 1, \cdots, n.$$

- So (b, w) is invariant under scaling.
- One more constraint $\min_{i=1}^n |b + x_i^T w| = 1$

Maximal Margin Classifier: Algorithm

- The problem becomes

$$\max_{b,w} \frac{1}{\sqrt{w^T w}}$$
$$\text{subject to} \qquad y_i(b + x_i^T w) \geq 1 \text{ for any } i = 1, \cdots, n.$$

- Or equivalently

$$\min_{b,w} \frac{1}{2} w^T w$$
$$\text{subject to} \qquad y_i(b + x_i^T w) \geq 1 \text{ for any } i = 1, \cdots, n.$$

- This is a QP!

Non-separable Data

- Maximal Margin Classifier: requires the data are separable!
- But not all data are separable!



- The above data are not separable by a linear boundary.
- This is often the case in practice.

Soft Margin

- we might be willing to consider a classifier based on a hyperplane that does not perfectly separate the two classes, to achieve
  - Greater robustness to individual observations,
  - Better classification of most of the training observations.



- The Maximal Margin Classifier (above figure) ensures
  - every observation is on the correct side of the hyperplane;
  - every observation is every observation is on the correct side of the hyperplane;
  - This is called hard margin.
- A soft margin classifier allows both above rules to be violated by some training observations (but not too many!)

Support Vector Classifier

- The following figure give examples of support vector classifier:
  - Most of the observations are on the correct side of the margin;
  - However, a small subset of the observations are on the wrong side of the margin (left panel).
  - An observation can be not only on the wrong side of the margin, but also on the wrong side of the hyperplane (right panel), which are mis-classified.
  - In fact, when there is no separating hyperplane, such a situation is inevitable.



Support Vector Classifier: Algorithm

- Maximal Margin Classifier:

$$\min_{b,w} \frac{1}{2} w^T w$$
$$\text{subject to} \qquad y_i(b + x_i^T w) \geq 1 \text{ for any } i = 1, \cdots, n.$$

- Support Vector Classifier:

$$\min_{b,w,\zeta_i} \frac{1}{2} w^T w + C \sum_{i=1}^{n} \zeta_i$$
$$\text{subject to} \qquad y_i(b + x_i^T w) \geq 1 - \zeta_i \text{ for any } i = 1, \cdots, n,$$
$$\zeta_i \geq 0 \text{ for any } i = 1, \cdots, n.$$

- ζ_i is called slack variables:
  - $\zeta_i = 0$: the ith data is on the correct side of the margin;
  - $\zeta_i > 0$: the ith data is on the wrong side of the margin;
  - $\zeta_i > 1$: the ith data is on the wrong side of the hyperplane (mis-classified);
- C is a tuning parameter (like λ in lasso) controlling how many training samples can be mis-classified.

Support Vector Classifier: Algorithm

- Example with different ζ_i:



- C is treated as a tuning parameter that is generally chosen via cross-validation.

Support Vector Classifier: Algorithm

- Lagrangian multiplier:

$$L = \frac{1}{2}w^T w + \sum_{i=1}^n \left\{ u_i[1 - \zeta_i - y_i(b + x_i^T w)] - v_i\zeta_i + C\zeta_i \right\} \text{ with } u_i, v_i \geq 0$$

- KKT conditions gives

$$\begin{aligned} w - \sum_{i=1}^n u_i y_i x_i = 0, \quad \sum_{i=1}^n u_i y_i = 0, \\ C - u_i - v_i = 0, \quad u_i \geq 0, \quad v_i \geq 0, \\ u_i[1 - \zeta_i - y_i(b + x_i^T w)] = 0, \quad v_i\zeta_i = 0. \end{aligned}$$

- So it follows that

$$\begin{aligned} L &= \frac{1}{2}w^T w + \sum_{i=1}^n u_i + \sum_{i=1}^n \zeta_i(C - u_i - v_i) - b\sum_{i=1}^n u_i y_i - \sum_{i=1}^n u_i y_i x_i^T w \\ &= -\frac{1}{2}\sum_{i=1}^n \sum_{j=1}^n u_i u_j y_i y_j x_i^T x_j + \sum_{i=1}^n u_i \end{aligned}$$

subject to $\sum_{i=1}^n u_i y_i = 0$ and $0 \leq u_i \leq C$ for $i = 1, \ldots, n$.

Support Vector Classifier: Algorithm

- Primal Problem:

$$\min_{b,w,\zeta_i} \frac{1}{2} w^T w + C \sum_{i=1}^{n} \zeta_i$$

$$\text{subject to} \quad y_i(b + x_i^T w) \geq 1 - \zeta_i \text{ for any } i = 1, \cdots, n,$$

$$\zeta_i \geq 0 \text{ for any } i = 1, \cdots, n.$$

- Dual Problem:

$$\max_{u_i} \left(-\frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} u_i u_j y_i y_j x_i^T x_j + \sum_{i=1}^{n} u_i \right)$$

$$\text{subject to} \quad \sum_{i=1}^{n} u_i y_i = 0,$$

$$0 \leq u_i \leq C \text{ for any } i = 1, \cdots, n.$$

- Both are QP!
- Primal problem has $p + 1$ parameters, dual has n.

From Dual to Primal

- From dual to primal: we solve the dual problem with solution u_i^*.
- Recall KKT conditions:

$$w - \sum_{i=1}^{n} u_i y_i x_i = 0, \quad \sum_{i=1}^{n} u_i y_i = 0,$$
$$C - u_i - v_i = 0, \quad u_i \geq 0, \quad v_i \geq 0,$$
$$u_i[1 - \zeta_i - y_i(b + x_i^T w)] = 0, \quad v_i \zeta_i = 0.$$

- The solution for primal problem is
  - $w^* = \sum_{i=1}^{n} u_i^* y_i x_i$
  - find any j such that $0 < u_j^* < C$, so $v_j^* > 0, \zeta_j^* = 0$, we can calculate
$$b^* = \frac{1}{y_j} - x_j^T w^*$$

Why Bother with Dual?

- n parameters in dual problem.
- $p+1$ parameters in primal problem.
- when $p+1 > n$, dual problem is easier to solve.
- Recall the dual problem:

$$\max_{u_i} \left(-\frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} u_i u_j y_i y_j x_i^T x_j + \sum_{i=1}^{n} u_i \right)$$

$$\text{subject to} \qquad \sum_{i=1}^{n} u_i y_i = 0, \text{ and } 0 \le u_i \le C \text{ for any } i = 1, \cdots, n.$$

- Kernel Trick: for kernel function $R(\cdot,\cdot) : \mathbb{R}^p \times \mathbb{R}^p \to \mathbb{R}$, define

$$\max_{u_i} \left(-\frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} u_i u_j y_i y_j R(x_i, x_j) + \sum_{i=1}^{n} u_i \right)$$

$$\text{subject to} \qquad \sum_{i=1}^{n} u_i y_i = 0, \text{ and } 0 \le u_i \le C \text{ for any } i = 1, \cdots, n.$$

Reproducing Kernel

- A function $R(\cdot,\cdot) : \mathbb{R}^p \times \mathbb{R}^p \to \mathbb{R}$ is called a reproducing kernel if
  $\sum_{i=1}^m \sum_{j=1}^m c_i c_j R(x_i, x_j) \geq 0$, for all m and $c_i, c_j \in \mathbb{R}$ and $x_i, x_j \in \mathbb{R}^p$.
  The equation holds if and only if $c_i = 0$ for all $i = 1, \ldots, m$.
- Feature map: there a function $\phi : \mathbb{R}^p \to \Omega$ such that
  $$R(x, \widetilde{x}) =< \phi(x), \phi(\widetilde{x}) >,$$
  where Ω is a Hilbert space with inner product $< \cdot, \cdot >$.
- Overall, the kernel trick replaces $x_i^T x_j$ by $R(x_i, x_j) =< \phi(x_i), \phi(x_j) >$

Reproducing Kernel

- Recall in linear svm:

$$f(x) = b + x^T w = b + x^T \sum_{i=1}^n u_i y_i x_i = b + \sum_{i=1}^n u_i y_i x^T x_i$$

- In kernel svm:

$$f(x) = b + \sum_{i=1}^n u_i y_i R(x, x_i)$$

- For fixed x_i, $R(x, x_i)$ is not necessary to be a linear function. Therefore, instead of separating hyperplane, kernel SVM only has a curved separating set.

Popular Kernels

- Linear kernel: $R(x, \widetilde{x}) = x^T \widetilde{x}$
  - Feature map is $\phi(x) = x$.
- Polynomial kernel with degree d: $R(x, \widetilde{x}) = (c + x^T \widetilde{x})^d$
  - Let $x = (x_1, x_2)^T \in \mathbb{R}^2$ and $\phi(x) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2)^T$.
  - So $\phi(x) : \mathbb{R}^2 \to \Omega$ with $\Omega = \mathbb{R}^3$ and

$$\begin{aligned} < \phi(x), \phi(\widetilde{x}) > &= (x_1^2, x_2^2, \sqrt{2} x_1 x_2)^T (\widetilde{x}_1^2, \widetilde{x}_2^2, \sqrt{2} \widetilde{x}_1 \widetilde{x}_2) \\ &= (x_1 \widetilde{x}_1 + x_2 \widetilde{x}_2)^2 = (0 + x^T \widetilde{x})^2. \end{aligned}$$

- Radial basis function (RBF) kernel: $R(x, \widetilde{x}) = e^{-\frac{\|x - \widetilde{x}\|^2}{\sigma^2}}$
  - The feature space where ϕ lies is infinite dimensional (no explicit formula).
- Sobolev kernel with degree 3:

$$\begin{aligned} R(x, \widetilde{x}) &= (1 + x\widetilde{x}) + \tfrac{1}{4} \left[(\widetilde{x} - \tfrac{1}{2})^2 - \tfrac{1}{12} \right] \left[(x - \tfrac{1}{2})^2 - \tfrac{1}{12} \right] \\ &\quad - \tfrac{1}{24} \left[(|x - \widetilde{x}| - \tfrac{1}{2})^4 - \tfrac{1}{2} (|x - \widetilde{x}| - \tfrac{1}{2})^2 + \tfrac{7}{240} \right]. \end{aligned}$$

  - Works only for $x \in \mathbb{R}$, recall smoothing spline.
  - When $x \in \mathbb{R}^p$, need tensor product kernel. For example:

$$R_2((x_1, x_2), (\widetilde{x}_1, \widetilde{x}_2)) = R(x_1, \widetilde{x}_1) R(x_2, \widetilde{x}_2) \text{ for } p = 2.$$

Which Kernel to Use?

- No kernel is better than the others uniformly.
- Personal favorite (other than linear kernel) is Gaussian RBF kernel.
- In low-dimensional data, if the separating boundary seems to be nonlinear, then kernel SVM usually works better (for prediction purpose.)
- However, for high-dimensional data $(d >> n)$, linear SVM is sufficient. Use the simpler linear SVM and avoid the complicated nonlinear SVM. The latter over-complicates the problem unnecessarily.
- Linear SVM gives better interpretations.

Heart Disease Data

```
library(kmed)
library(e1071)
set.seed(1)
data=heart
data$class[which(data$class>0)]=1
data$class=factor(data$class)
dim(data)

 [1] 297  14

index=sample(1:297,60)
testdata=data[index,]
traindata=data[setdiff(1:297,index),]


tune.out=tune(svm, class~., data=traindata, kernel ="linear",
ranges=list(cost=exp(seq(-10,0, length.out=20))))

plot(tune.out$performances[,1], tune.out$performances[,2],
type="l", xlab="cost",ylab="CV")
```

Heart Disease Data

- CV with different cost values:




- ```
  bestmod=tune.out$best.model
  yhat=predict(bestmod,testdata)
  y=testdata$class
  mean(yhat==y)
  ```

  ```
  [1] 0.8166667
  ```
  ```

K-class Classification

- So far, our discussion has been limited to the case of binary classification: two-class setting.
- How can we extend SVMs to general K classes?
- There are two popular approach
  - One-Versus-One:
    - Use only data from class k and l: construct a binary classier with a score function $\hat{f}_{kl}(x)$, $k, l = 1, \cdots, K$.
    - For a new feature vector x^*, use the fitted $\binom{K}{2}$ SVM classier to assign x^* to each of the K-classes;
    - x^* is finally classified to k^* if x^* is assigned to class k^* most of times (out of $\binom{K}{2}$ times)
  - One-Versus-All:
    - Use only data from class k and combine data from all other classes into a bigger class, say class A;
    - For each k, construct a binary classier with a score function $\hat{f}_k(x)$, $k, l = 1, \cdots, K$.
    - For a new feature vector x^*, compute scores for $\hat{f}_k(x^*)$, $k, l = 1, \cdots, K$.
    - x^* is finally classified to k^* if $\hat{f}_k(x^*)$ is the largest!
  - Which one to use? If K is not too large, use One-Versus-One!

Mis-classification Risk and Bayesian Classifier

- Consider the two categorical classification problem with $Y \in \{-1, 1\}$ and $\boldsymbol{X} \in \mathbb{R}^p$.
- Given a classifier $f : \mathbb{R}^p \to \{-1, 1\}$, its mis-classification risk is defined as
$$\mathcal{E}(f) = P(f(\boldsymbol{X}) \neq Y).$$
- What is the best classifier minimizes the mis-classification risk?
- Define $p_{+1}(x) = P(Y = 1|\boldsymbol{X} = x)$ and $p_{-1}(x) = P(Y = -1|\boldsymbol{X} = x)$.
- Bayesian Classifier: $f^*(x) = \begin{cases} 1 & \text{if } p_{+1}(x) \geq p_{-1}(x) \\ -1 & \text{if } p_{+1}(x) < p_{-1}(x) \end{cases}$ is the best.
- **Proof:** Let $g(x)$ be the density of X, then

$$\begin{aligned} \mathcal{E}(f) &= P(f(\boldsymbol{X}) \neq Y) \\ &= P(f(\boldsymbol{X}) \neq Y, Y = 1) + P(f(\boldsymbol{X}) \neq Y, Y = -1) \\ &= P(f(\boldsymbol{X}) = -1, Y = 1) + P(f(\boldsymbol{X}) = 1, Y = -1) \end{aligned}$$

Mis-classification Risk and Bayesian Classifier

- Let $g(x)$ be the density of X and by conditional probability formula, we have

$$\begin{aligned} P(f(\boldsymbol{X}) = -1, Y = 1) &= \int P(f(X) = -1, Y = 1 | X = x) g(x) dx \\ &= \int P(f(x) = -1, Y = 1 | X = x) g(x) dx. \end{aligned}$$

- Notice that

$$P(f(x) = -1, Y = 1 | X = x) = \begin{cases} P(Y = 1 | X = x) = p_{+1}(x) & \text{if } f(x) = -1 \\ 0 & \text{if } f(x) = 1 \end{cases}$$

- So $P(f(\boldsymbol{X}) = -1, Y = 1) = \int_{f(x)=-1} p_{+1}(x) g(x) dx$
- Similarly, we can show $P(f(\boldsymbol{X}) = 1, Y = -1) = \int_{f(x)=1} p_{-1}(x) g(x) dx$

Mis-classification Risk and Bayesian Classifier

- Notice $p_{+1}(\boldsymbol{X}) + p_{-1}(\boldsymbol{X}) = 1$ and $f(x) \in \{-1, 1\}$, so

$$
\begin{aligned}
\mathcal{E}(f) &= P(f(\boldsymbol{X}) \neq Y) \\
&= P(f(\boldsymbol{X}) = -1, Y = 1) + P(f(\boldsymbol{X}) = 1, Y = -1) \\
&= \int_{f(x)=-1} p_{+1}(x) g(x) dx + \int_{f(x)=1} p_{-1}(x) g(x) dx \\
&= \int_{f(x)=-1} p_{+1}(x) g(x) dx - \int_{f(x)=-1} p_{-1}(x) g(x) dx \\
&\quad + \int p_{-1}(x) g(x) dx \\
&= \int_{f(x)=-1} [2p_{+1}(x) - 1] g(x) dx + \underbrace{\int p_{-1}(x) g(x) dx}_{\text{free of } f}.
\end{aligned}
$$

- Therefore, we have

$$
\begin{aligned}
&\mathcal{E}(f) - \mathcal{E}(f^*) \\
= &\int_{f(x)=-1} [2p_{+1}(x) - 1] g(x) dx - \int_{f^*(x)=-1} [2p_{+1}(x) - 1] g(x) dx
\end{aligned}
$$

Mis-classification Risk and Bayesian Classifier

- For any x, we have two cases.
- Case 1: $x \in A = \{2p_{+1}(x) \geq 1\}$.
- In this case, we have $A = \{p_{+1}(x) \geq 1/2\}$ and $A = \{f^*(x) = 1\}$.
- Therefore, we have

$$
\begin{aligned}
& \int_{f(x)=-1, x\in A} [2p_{+1}(x) - 1]g(x)dx - \int_{f^*(x)=-1, x\in A} [2p_{+1}(x) - 1]g(x)dx \\
= & \int_{f(x)=-1, f^*(x)=1} [2p_{+1}(x) - 1]g(x)dx \\
= & \int_{f(x)\neq f^*(x), f^*(x)=1} |2p_{+1}(x) - 1|g(x)dx
\end{aligned}
$$

Mis-classification risk and Bayesian classifier

- Case 2: $x \in A^c = \{2p_{+1}(x) < 1\}$.
- In this case, we have $A^c = \{p_{+1}(x) < 1/2\}$ and $A^c = \{f^*(x) = -1\}$.
- Therefore, we have

$$
\begin{aligned}
&\int_{f(x)=-1, x\in A^c} [2p_{+1}(x) - 1]g(x)dx - \int_{f^*(x)=-1, x\in A^c} [2p_{+1}(x) - 1]g(x)dx \\
&= \int_{f(x)=-1, f^*(x)=-1} [2p_{+1}(x) - 1]g(x)dx - \int_{f^*(x)=-1} [2p_{+1}(x) - 1]g(x)dx \\
&= -\int_{f(x)=1, f^*(x)=-1} [2p_{+1}(x) - 1]g(x)dx \\
&= -\int_{f(x)\neq f^*(x), f^*(x)=-1} [2p_{+1}(x) - 1]g(x)dx \\
&= \int_{f(x)\neq f^*(x), f^*(x)=-1} |2p_{+1}(x) - 1|g(x)dx.
\end{aligned}
$$

Mis-classification Risk and Bayesian Classifier

- Recall

$$
\begin{aligned}
& \mathcal{E}(f) - \mathcal{E}(f^*) \\
= \quad & \int_{f(x)=-1} [2p_{+1}(x) - 1] g(x) dx - \int_{f^*(x)=-1} [2p_{+1}(x) - 1] g(x) dx
\end{aligned}
$$

- Combining above arguments, we have

$$
\begin{aligned}
& \mathcal{E}(f) - \mathcal{E}(f^*) \\
= & \int_{f(x)=-1} - \int_{f^*(x)=-1} \\
= & \int_{f(x)=-1, x\in A} + \int_{f(x)=-1, x\in A^c} - \int_{f^*(x)=-1, x\in A} - \int_{f^*(x)=-1, x\in A^c} \\
= & \int_{f(x)\neq f^*(x), f^*(x)=1} |2p_{+1}(x) - 1| g(x) dx + \int_{f(x)\neq f^*(x), f^*(x)=-1} |2p_{+1}(x) - 1| = g(x) dx \\
= & \int_{f(x)\neq f^*(x)} |2p_{+1}(x) - 1| g(x) dx \geq 0.
\end{aligned}
$$

K-class Bayesian classifier

- Similarly, when there are K classes, the corresponding Bayesian classifier is

$$f^*(x) = k \text{ if } p_k(x) \geq \max_{1 \leq i \leq K} p_i(x),$$

  where $p_i = P(Y = i | \boldsymbol{X} = x)$ for $i = 1, \ldots, K$.
- There could be multiple indexes achieve the maximal, so there could be several Bayesian classifiers.
- However, Bayesian classifier is not feasible in practice.
- We do not know p_i's, which rely on the distribution of $(\boldsymbol{X}, Y)$.
- But we can estimate p_i's!

Linear Discriminant Analysis (LDA): $p = 1$

- We first consider the case when the feature vector X is univariate;
- The difficulty with classification lies in estimating p_k's!
- The easiest way is to make assumptions!
- LDA assumes conditional densities $f_k(\cdot) = f(x|Y = k)$'s are normal.

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x - \mu_k)^2\right), \quad k = 1, \cdots, K$$

- LDA assumes that all variances are the same $\sigma_1^2 = \cdots = \sigma_K^2 = \sigma^2$;
- LDA assumes $\pi_k = P(Y = k)$ for $k = 1, \ldots, K$
- Under the normal assumption, the classification probability becomes

$$p_k(x) = P(Y = k|X = x) = \frac{\pi_k \exp\left(-\frac{1}{2\sigma^2}(x - \mu_k)^2\right)}{\sum_{l=1}^{K} \pi_l \exp\left(-\frac{1}{2\sigma^2}(x - \mu_l)^2\right)}, \quad k = 1, \cdots, K.$$

- The Bayes classifier assigns the observation to class k with the largest $p_k(x)$, or, equivalently, the largest

$$\delta_k(x) = x\frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k)$$

LDA: $p = 1$

- When there are only two categories $K = 2$;
- We have two scores to compare with

$$\delta_k(x) = x\frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k), \quad k = 1, 2;$$

- Consider the case when $\pi_1 = \pi_2$, then an observation will be assigned to Class 1 if

$$\delta_1(x) > \delta_2(x) \Rightarrow x(\mu_1 - \mu_2) > \frac{1}{2}(\mu_1^2 - \mu_2^2)$$

  - If $\mu_1 > \mu_2$, the rule becomes $x > \frac{\mu_1+\mu_2}{2}$;
  - If $\mu_1 < \mu_2$, the rule becomes $x < \frac{\mu_1+\mu_2}{2}$;
- Therefore, the decision boundary for the Bayes classifier becomes

$$x = \frac{\mu_1 + \mu_2}{2},$$

which is obtained by solving $\delta_1(x) = \delta_2(x)$.

LDA: $p = 1$

- When there are only two categories $K = 2$;
- Data were simulated from
  - For group 1, $n_1 = 20$ points from $N(-1.25, 1^2)$ (i.e. $\mu_1 = -1.26$, $\sigma^2 = 1$);
  - For group 2, $n_2 = 20$ points from $N(1.25, 1^2)$ (i.e. $\mu_2 = 1.26$, $\sigma^2 = 1$);
- In practice, π_k's, μ_k's and σ^2 are unknown.
- Estimate the membership probability: $\hat{\pi}_1 = \frac{n_1}{n_1+n_2} = 1/2, \hat{\pi}_2 = \frac{n_2}{n_1+n_2} = 1/2$;
- Estimate sample means and sample variances
  - Group sample means: $\hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} x_i,$ for $k = 1, 2$;
  - Pooled variance: $\hat{\sigma}^2 = \frac{1}{n-2} \sum_{k=1}^{2} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)^2$, where $n = n_1 + n_2$;
- Compute the estimated score for each group:

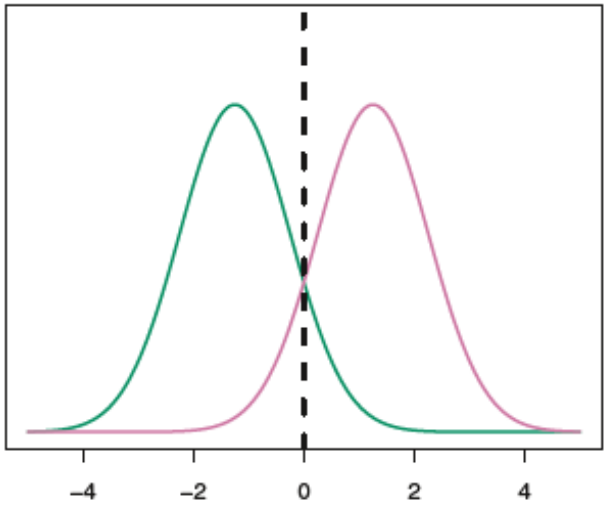
$$\hat{\delta}_k(x) = x\frac{\hat{\mu}_k}{\hat{\sigma}^2} - \frac{\hat{\mu}_k^2}{2\hat{\sigma}^2} + \log(\hat{\pi}_k), \quad k = 1, 2;$$

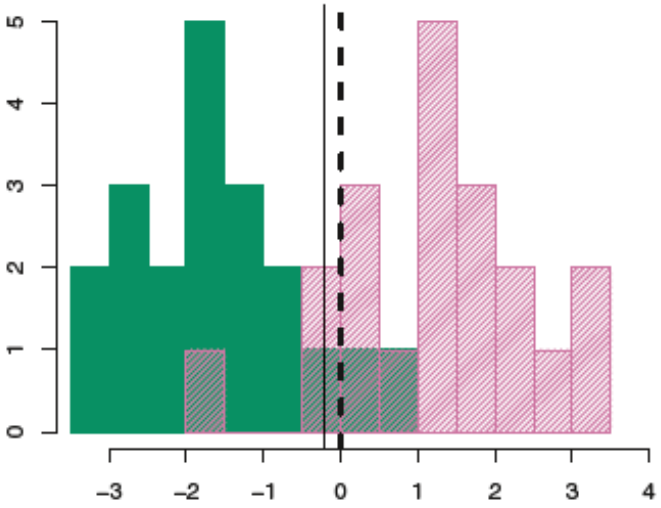
- Assign an observation with $X = x$ to class 1 if $\hat{\delta}_1(x)$ is larger, otherwise to class 2;

LDA: $p = 1$

- The Bayes classier decision boundary is obtained by solving $\delta_1(x) = \delta_2(x)$, which gives
$$x = \frac{\mu_1 + \mu_2}{2} = \frac{-1.25 + 1.25}{2} = 0,$$
- The LDA classier decision boundary is obtained by solving $\hat{\delta}_1(x) = \hat{\delta}_2(x)$, which gives
$$x = \frac{\hat{\mu}_1 + \hat{\mu}_2}{2} \neq 0,$$
- What's the difference? Which one is better?

LDA: $p = 1$





- The solid vertical line: the estimated LDA boundary;
- All points to the left of this line will be assigned to the green class;
- All points to the right of this line are assigned to the purple class.
- The LDA decision boundary is slightly to the left of the optimal Bayes decision boundary $x = 0$;
- After generating a large number of test observations we can compute the Bayes error rate (10.6%) and the LDA test error rate (11.1%).
- LDA is slightly worse than the optimal Bayes classifier!

LDA: Multivariate Feature Space $p > 1$

- When the feature vector X is multivariate, LDA assumes that its density follows multivariate normal $X|Y = k \sim N(\mu_k, \mathbf{\Sigma})$;

$$f_k(x) = \frac{1}{(2\pi)^{p/2}|\mathbf{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_k)^T \mathbf{\Sigma}^{-1}(x-\mu_k)\right), \quad k = 1, \cdots, K$$

  - LDA assume that the covariance matrix $\mathbf{\Sigma}$ are the same for all classes;
- The Bayes classifier assigns the observation to class k with the largest $\pi_k f_k(x)$, or, equivalently, the largest

$$\delta_k(x) = x^T \mathbf{\Sigma}^{-1}\mu_k - \frac{1}{2}\mu_k^T \mathbf{\Sigma}^{-1}\mu_k + \log(\pi_k)$$

- The above scores $\delta_k(x)$'s are linear functions of x, hence the name LDA (Linear Discriminant Analysis)

LDA: Multivariate Feature Space $p > 1$

- When there are three classes, i.e., $K = 3$;
- We have three score functions to compare with

$$\delta_k(x) = x^T \mathbf{\Sigma}^{-1} \mu_k - \frac{1}{2} \mu_k^T \mathbf{\Sigma}^{-1} \mu_k + \log(\pi_k), \quad k = 1, 2, 3;$$

- Consider the case when $\pi_1 = \pi_2 = \pi_3$, then an observation will be assigned to Class 1 if

$$\delta_1(x) > \delta_2(x) \text{ and } \delta_1(x) > \delta_3(x)$$

- When $K = 3$, we would have three the decision boundaries for the Bayes classifier by solving
  - $\delta_1(x) = \delta_2(x)$; $\delta_1(x) = \delta_3(x)$; and $\delta_2(x) = \delta_3(x)$;

  which gives the boundaries as

$$x^T \mathbf{\Sigma}^{-1} \mu_k - \frac{1}{2} \mu_k^T \mathbf{\Sigma}^{-1} \mu_k = x^T \mathbf{\Sigma}^{-1} \mu_l - \frac{1}{2} \mu_l^T \mathbf{\Sigma}^{-1} \mu_l$$

- the decision rule depends on x only through a linear combination of its elements!

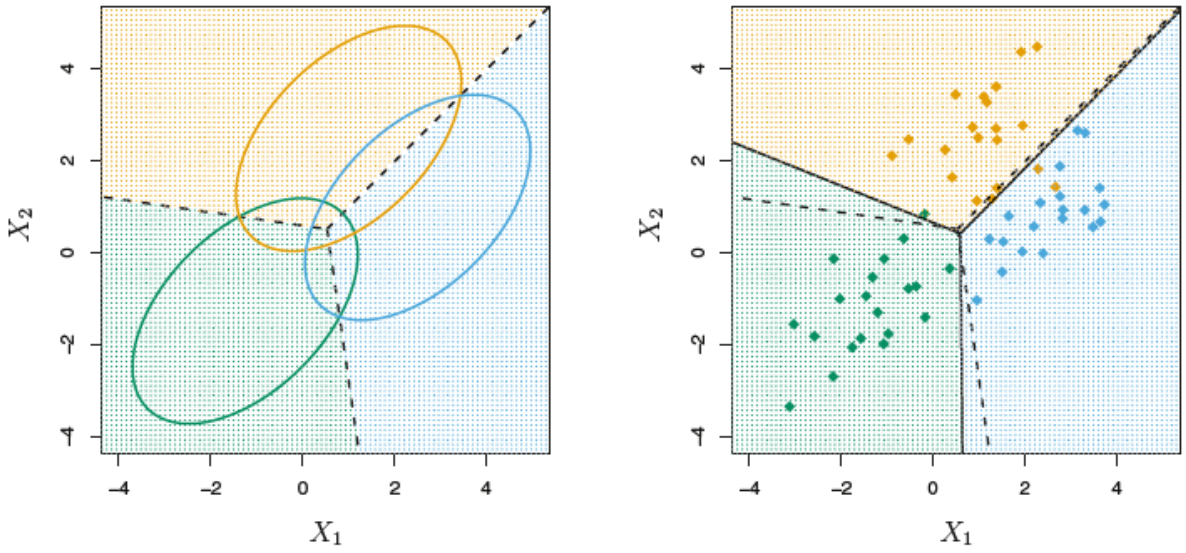
LDA: Multivariate Feature Space $p > 1$

- $n_k = 20$ were simulated from $p = 2$ dimensional normal distribution, $k = 1, 2, 3$;
- Estimate the membership probability: $\hat{\pi}_1 = \hat{\pi}_2 = \hat{\pi}_3 = 20/60 = 1/3$;
- Estimate sample means and sample covariance matrix
  - Group sample means: $\hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} x_i$, for $k = 1, 2, 3$;
  - Pooled covariance matrix: $\widehat{\mathbf{\Sigma}} = \frac{1}{n-K} \sum_{k=1}^{K} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$, where $n = n_1 + \cdots + n_K$;
- Compute the estimated score for each group:

$$\hat{\delta}_k(x) = x^T \widehat{\mathbf{\Sigma}}^{-1} \mu_k - \frac{1}{2} \hat{\mu}_k^T \widehat{\mathbf{\Sigma}}^{-1} \hat{\mu}_k + \log(\hat{\pi}_k), \quad k = 1, 2, 3;$$

- Assign an observation with $X = x$ to class k if $\hat{\delta}_k(x)$ is the largest;

LDA: Multivariate Feature Space $p > 1$



- The dashed vertical line: Bayes classifier boundary;
- The solid vertical line: the estimated LDA boundary;
- Bayes (LDA) decision boundaries divide the feature space into three regions;
- The Bayes classifier (LDA) will classify an observation according to the region in which it is located.
- After generating a large number of test observations we can compute the Bayes error rate (7.46%) and the LDA test error rate (7.70%).

QDA: Multivariate Feature Space $p > 1$

- Assume that the feature vector X in each group follows multivariate normal distribution, i.e., $X|Y = k \sim N(\mu_k, \mathbf{\Sigma}_k)$;

$$f_k(x) = \frac{1}{(2\pi)^{p/2}|\mathbf{\Sigma}_k|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_k)^T \mathbf{\Sigma}_k^{-1}(x-\mu_k)\right), \quad k = 1, \cdots, K$$

  - LDA assumes that the covariance matrices $\mathbf{\Sigma}_1 = \cdots = \mathbf{\Sigma}_K$ are the same for all classes;
  - QDA assumes that the covariance matrices $\mathbf{\Sigma}_1, \cdots, \mathbf{\Sigma}_K$ are different for all classes;
- Under this assumption, the Bayes classifier assigns the observation to class k with the largest $\pi_k f_k(x)$, or, equivalently, the largest

$$\delta_k(x) = -\frac{1}{2}x^T \mathbf{\Sigma}_k^{-1} x + x^T \mathbf{\Sigma}_k^{-1}\mu_k - \frac{1}{2}\mu_k^T \mathbf{\Sigma}_k^{-1}\mu_k - \frac{1}{2}\log|\mathbf{\Sigma}_k| + \log(\pi_k)$$

- The above scores $\delta_k(x)$'s are quadratic functions of x, hence the name QDA (Quadratic Discriminant Analysis)

QDA: Multivariate Feature Space $p > 1$

- One still needs to estimate the membership probability: $\hat{\pi}_1 = \cdots = \hat{\pi}_K = n_k/n$;
- Estimate sample means and all covariance matrices
  - Group sample means: $\hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} x_i$, for $k = 1, \cdots, K$;
  - Covariance matrices: $\widehat{\mathbf{\Sigma}}_k = \frac{1}{n_k - 1} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$, for $k = 1, \cdots, K$;
- Compute the estimated score for each group:

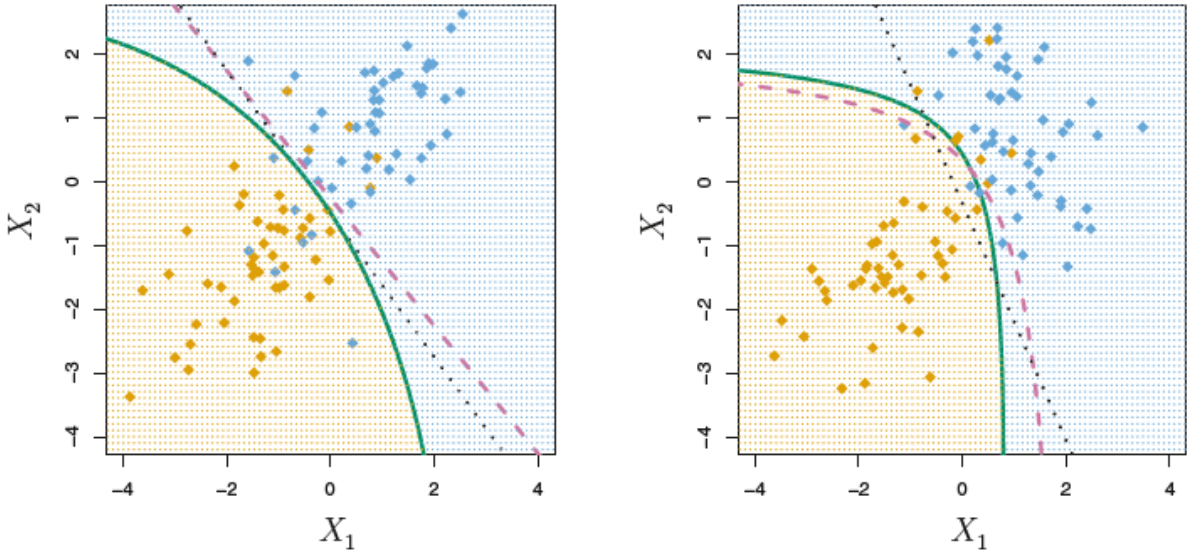
$$\hat{\delta}_k(x) = -\frac{1}{2} x^T \widehat{\mathbf{\Sigma}}_k^{-1} x + x^T \widehat{\mathbf{\Sigma}}_k^{-1} \hat{\mu}_k - \frac{1}{2} \hat{\mu}_k^T \widehat{\mathbf{\Sigma}}_k^{-1} \hat{\mu}_k - \frac{1}{2} \log |\widehat{\mathbf{\Sigma}}_k| + \log(\hat{\pi}_k)$$

- Assign an observation with $X = x$ to class k if $\hat{\delta}_k(x)$ is the largest;

LDA v.s. QDA

- Advantages of QDA:
  - The QDA is theoretically superior to the LDA because it imposes less restrictive assumptions;
  - In fact, LDA is a special case of QDA;
- Why not always use QDA?
  - Theory is just theory (:( ); in practice, many things need to be estimated using data;
  - QDA requires estimating much more parameters, including $\mathbf{\Sigma}_1, \cdots, \mathbf{\Sigma}_K$ and means μ_k's;
  - LDA only needs to estimate one common covariance matrix $\mathbf{\Sigma}$ and means μ_k's!
- When to use LDA and QDA?
  - QDA needs more data! So if you have a large sample size, you can consider use QDA;
  - LDA should be used if you only have a limited sample size;
  - QDA is better when (1) you have more data; and (2) the decision boundaries are not linear!
  - Otherwise, it is better to use QDA;

LDA V.S. QDA: Example



- Left plot: The Bayes (purple dashed), LDA (black dotted), and QDA (green solid) decision boundaries for a two-class problem with $\mathbf{\Sigma}_1 = \mathbf{\Sigma}_2$;
- The shading indicates the QDA decision rule.
  - Since the Bayes decision boundary is linear, it is more accurately approximated by LDA than by QDA.
- Right plot: a two-class problem with $\mathbf{\Sigma}_1 \neq \mathbf{\Sigma}_2$;
  - Since the Bayes decision boundary is non-linear, it is more accurately approximated by QDA than by LDA.

LDA v.s. QDA: R implementation

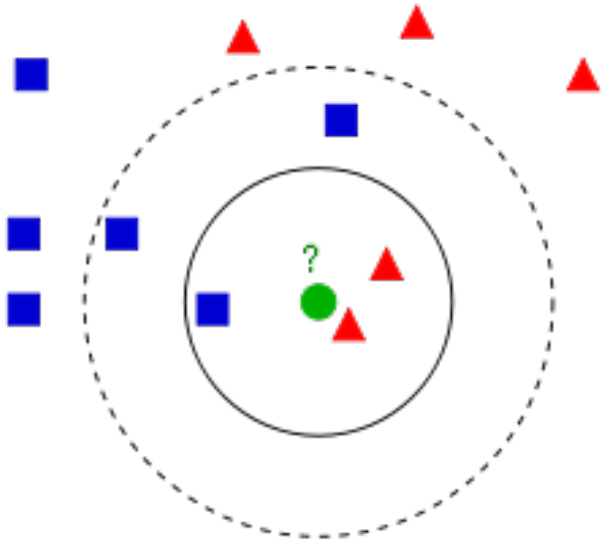
- The implementation of LDA and QDA are very easy using the R package "MASS";
- Use function *lda()* for LDA;
- Use function *qda()* for QDA;

Averaging V.S. Voting

- The idea of LDA and QDA is to compare the conditional probability $p_k(x) = P(Y = k|\boldsymbol{X} = x)$.
- Limitation: the normal assumption on X given $Y = k$ is too strong.
- Idea: directly estimate $p_k(x)$.
- Can we use spline linear regression?
- Yes, when $K = 2$.
- No, when $K > 2$.
- No. For example, coding $Y = 1, 2, 3$ and $Y = 0, 1, 10000$ may give you different results.
- Regression basically is taking weighted average of $y_i's$. (Recall hat matrix)
- We can replace averaging with voting.

Nearest Neighbor Classifier

- Step 1: For point x to be classified, we find the k nearest points to x among all the covariates $x_1, x_2, \ldots, x_n$.
- Step 2: count the number of each categories among these k observations.
- Step 3: classify x to be the category with the most count.
- Example of k-NN with two classes and $k = 3, 5$:



- When there is a tie, just randomly pick one class.

Nearest Neighbor Classifier

- For point x to be classified, we denote the k nearest points to x are

$$x_{(1)}, x_{(2)}, \ldots, x_{(k)}.$$

- Moreover, we can write the associated labels as $y_{(1)}, y_{(2)}, \ldots, y_{(k)}$.
- Remember, $(x_{(i)}, y_{(i)})$'s are depending on x.
- The conditional probability $p_s(x) = P(Y = s|\boldsymbol{X} = x)$ is estimated by

$$\widehat{p}_s(x) = \frac{1}{k}\sum_{i=1}^{k} I(y_{(i)} = s).$$

- Finally, we pick the class with the largest $\widehat{p}_s(x)$.

Nearest Neighbor Classifier

- How to choose k?
- Cross validation!
- Magnitude issue: suppose x contains age and annual income (in dollar), which one will have more impacts on classification result?
- Example: a 20-years-old with 10k income and a 40-years-old with 20k income.
- The distance is almost determined by income only!
- To reduce the influence of the magnitude, all variables should be standardized to a comparable scale.
- For example, to $[0, 1]$ or transform to variables with mean zero and variance 1.
- R package "class": knn(train, test, cl, k = 1, prob = FALSE)

Caravan Insurance Data

```
library(ISLR)
library(caret)
library(class)
set.seed(1)
x=scale(Caravan [,-86])
y=Caravan[,86]
data=data.frame(x)
data$y=y
trControl=trainControl(method  = "cv", number  = 5)

fit=train(y~.,
          method     = "knn",
          tuneGrid   = expand.grid(k = 1:20),
          trControl  = trControl,
          metric     = "Accuracy",
          data       = data)

fit
```

Caravan Insurance Data

```
k-Nearest Neighbors

5822 samples
  85 predictor
   2 classes: 'No', 'Yes'

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 4657, 4657, 4657, 4658, 4659
Resampling results across tuning parameters:

  k   Accuracy   Kappa
   1  0.8936778   0.0695236437
   2  0.8919596   0.0429053029
   3  0.9268286   0.0604255975
   4  0.9290630   0.0542062554
   5  0.9359341   0.0378326336
   6  0.9357618   0.0373906839
   7  0.9390251   0.0266643110
   8  0.9397124   0.0281622681
   9  0.9395404   0.0134898343
  10  0.9395403   0.0134272717
  11  0.9403989   0.0203055869
  12  0.9398838   0.0043752237
  13  0.9398835  -0.0006783938
  14  0.9400553  -0.0003390857
  15  0.9402270   0.0000000000
  16  0.9400553   0.0045652205
  17  0.9402270   0.0000000000
  18  0.9402270   0.0000000000
  19  0.9402270   0.0000000000
  20  0.9402270   0.0000000000

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was k = 11.
```

Caravan Insurance Data

```
standardized.X=scale(Caravan[,-86])
test =1:1000
train.X=standardized.X[-test,]
test.X= standardized.X[test,]
train.Y=y[-test]
test.Y=y[test]
set.seed (1)
knn.pred=knn(train=train.X,test=test.X, cl=train.Y,k=11)

mean(test.Y==knn.pred)

[1] 0.941
```

Loss Function Method – Binary Cross Entropy Loss

- Two classes: $Y_i = \{0, 1\}$
- Conditional probability: $p(x) = P(Y = 1|X = x)$
- Classifier:
  - $\widehat{y}_p(x) = 1$ if $p(x) > 1/2$
  - $\widehat{y}_p(x) = 0$ if $p(x) \leq 1/2$
- Binary cross entropy (BCE) loss:

$$
\begin{aligned}
L(y, p(x)) &= -\big(y \log(p(x)) + (1-y)\log(1-p(x))\big) \\
&= \begin{cases} -\log(p(x)), & \text{if } y = 1 \\ -\log(1-p(x)) & \text{if } y = 0 \end{cases}
\end{aligned}
$$

- Data: $(X_i, Y_i), i = 1, \ldots, n$
- Training:

$$
\widehat{p} = \underset{p \in \mathcal{P}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} L(Y_i, p(X_i)).
$$

- $\mathcal{P}$: class of functions

Loss Function Method – Binary Cross Entropy Loss

- We need $p(x) \in [0, 1]$
- If a number $u \in \mathbb{R}$, use the sigmoid transformation:

$$v = Sigmoid(u) = \frac{1}{1 + e^{-u}}$$

- So that $v \in [0, 1]$
- Compare BCELoss and BCEWithLogitsLoss in Pytorch

Loss Function Method – Cross Entropy Loss

- K classes: $Y_i = \{0, 1, \ldots, K-1\}$
- Conditional probability: $p(x) = (p_0(x), \ldots, p_{K-1}(x))$
  - $p_k(x) = P(Y = k | X = x), k = 0, \ldots, K-1$
  - $p_0(x) + \ldots + p_{K-1}(x) = 1$
- Classifier: $\widehat{y}_p(x) = k$ if $p_k(x)$ is the largest
- Cross entropy loss:

$$L(y, p(x)) = -\log(p_k(x)) \text{ if } y = k.$$

- Data: $(X_i, Y_i), i = 1, \ldots, n$
- Training:

$$\widehat{p} = \underset{p \in \mathcal{P}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} L(Y_i, p(X_i)).$$

Loss Function Method – Cross Entropy Loss

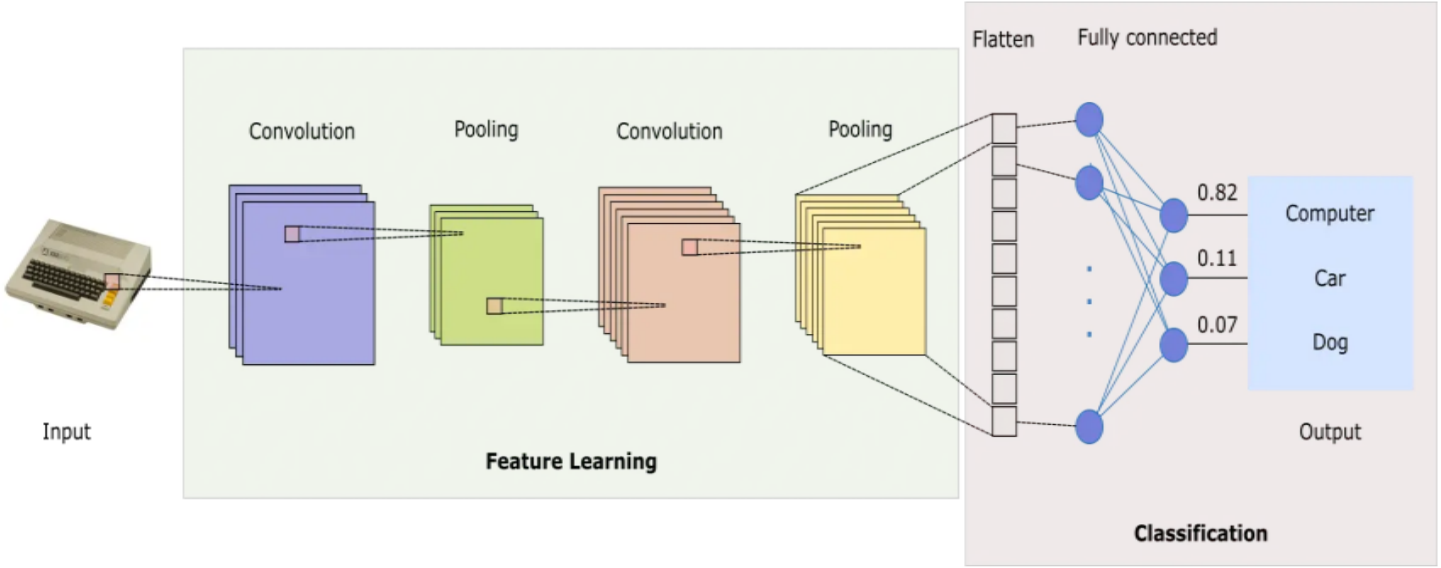
- $p(x)$ should satisfy $p_0(x) + \ldots + p_{K-1}(x) = 1$.
- If a vector $u = (u_0, \ldots, u_{K-1})$ does not satisfy this conditions, use the softmax transformation:

$$v_k = Softmax(u)_k = \frac{e^{u_k}}{\sum_{i=0}^{K-1} e^{u_i}}$$

- So that $v_0 + \ldots + v_{K-1} = 1$
- CrossEntropyLoss in Pytorch applies sofmax transformation automatically.

Convolutional Neural Networks

- CNN is designed for image classification
- Network structure of CNN:



Graphic Representation

- Pictures are matrices in Python
- Colored picture: $3 \times w \times h$ (R,G B)
- Grayscale picture: $1 \times w \times h$
- Each element is an integer in $[0, 255]$

Convolution Operation

- Convolution with 3×3 kernel:

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

- $0 \times 1 + 0 \times (-1) + 0 \times 2 + 0 \times 2 + 1 \times (-1) + 1 \times 3 + 0 \times (-1) + 3 \times 5 + 3 \times 0 = 17$

Convolution Operation

- Convolution with 3×3 kernel:

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

Convolution Operation

- Convolution with 3×3 kernel:

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

- Keep moving...

Convolution Operation

- Convolution with 3×3 kernel:

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

Padding

- Picture with size 4×4 with padding$= 1$:

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

Stride

- Convolution with Stride$= 1$:

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

Stride

- Convolution with Stride$= 2$:

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

Convolution Calculation

- Convolution with Stride$= 1$ (default in Pytorch):

0	0	0	0	0	0
0	1	1	2	1	0
0	3	3	2	3	0
0	1	5	3	2	0
0	1	4	5	4	0
0	0	0	0	0	0

1	-1	2
2	-1	3
-1	5	0

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

- Picture: $w \times w$ with padding p
- Kernel: $k \times k$
- Output: $o \times o$ with $o = (w + 2p - k + 1)$
- If $k = 2p + 1$, then input and output have same dimension
- For general Stride$= s$, we have $o = (w + 2p - k)/s + 1$

Max Pooling

- Pooling kernel: 2×2
- Max pooling with Stride=Kernel size (default in Pytorch):

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

37	24
29	41

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

37	24
29	41

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

37	24
29	41

17	19	10	16
12	37	24	9
22	29	41	18
20	15	21	7

37	24
29	41

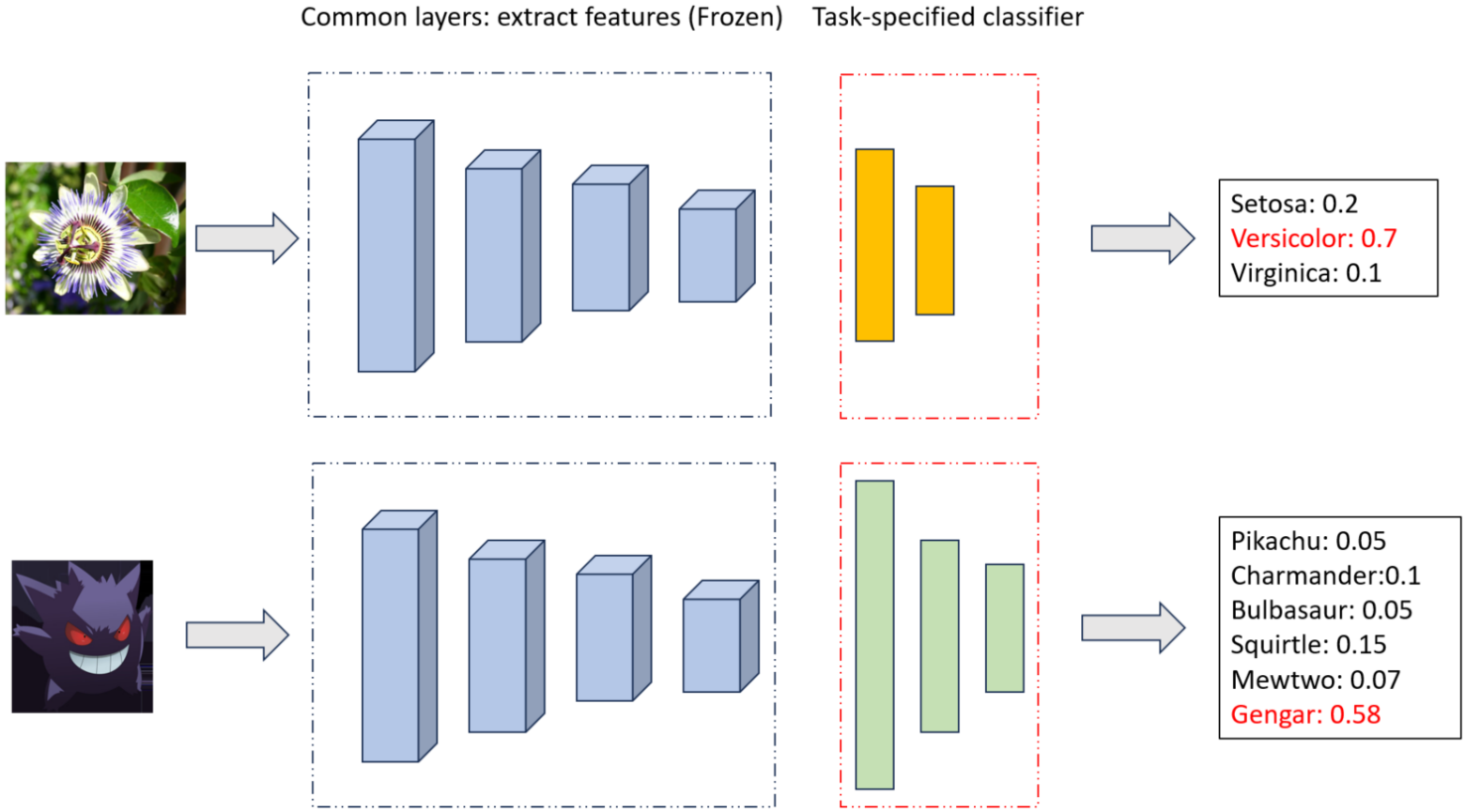
- $\text{Output dimension} = \frac{\text{Input dimension}}{\text{Kernel size}}$ (loosely speaking)

Coding

- See instruction in class

Transfer Learning

- Using information from pre-trained model:



- Pytorch: AlexNet, ResNet, VGG...

Coding

- See instruction in class